

Improving Resource Aware ML through Resource Decomposition

07-599 SCS Senior Thesis

Arnav Sabharwal, Advisor: Jan Hoffmann

Resource Aware ML (RaML)

What is RaML?

A tool ¹ for automatically bounding the worst-case cost of ML programs

¹[2]

Resource Aware ML (RaML)

What is RaML?

A tool ¹ for automatically bounding the worst-case cost of ML programs

```
let rec qsort l =  
  tick 1.0; match l with  
  | [] -> []  
  | x :: xs ->  
    let ys, zs = partition x xs in  
    append (qsort ys) (x :: qsort zs)
```

¹[2]

Resource Aware ML (RaML)

What is RaML?

A tool ¹ for automatically bounding the worst-case cost of ML programs

```
let rec qsort l =  
  tick 1.0; match l with  
  | [] -> []  
  | x :: xs ->  
    let ys, zs = partition x xs in  
    append (qsort ys) (x :: qsort zs)
```

RaML's bound:

$$1 + 3|l| + |l|^2$$

¹[2]

Resource Aware ML (RaML)

What is RaML?

A tool ¹ for automatically bounding the worst-case cost of ML programs

```
let rec qsort l =  
  tick 1.0; match l with  
  | [] -> []  
  | x :: xs ->  
    let ys, zs = partition x xs in  
    append (qsort ys) (x :: qsort zs)
```

RaML's bound:

$$1 + 3|l| + |l|^2$$

Challenges

RaML is unable to infer tight worst-case bounds on divide-and-conquer style programs

¹[2]

Resource Aware ML (RaML)

What is RaML?

A tool ¹ for automatically bounding the worst-case cost of ML programs

```
let rec qsort l =  
  tick 1.0; match l with  
  | [] -> []  
  | x :: xs ->  
    let ys, zs = partition x xs in  
    append (qsort ys) (x :: qsort zs)
```

RaML's bound:

$$1 + 3|l| + |l|^2$$

Challenges

RaML is unable to infer tight worst-case bounds on divide-and-conquer style programs

```
let rec msort l =  
  tick 1.0; match l with  
  | [] -> []  
  | _ :: _ ->  
    let l1, l2 = split l in  
    merge (msort l1) (msort l2)
```

¹[2]

Resource Aware ML (RaML)

What is RaML?

A tool ¹ for automatically bounding the worst-case cost of ML programs

```
let rec qsort l =  
  tick 1.0; match l with  
  | [] -> []  
  | x :: xs ->  
    let ys, zs = partition x xs in  
    append (qsort ys) (x :: qsort zs)
```

RaML's bound:

$$1 + 3|l| + |l|^2$$

Challenges

RaML is unable to infer tight worst-case bounds on divide-and-conquer style programs

```
let rec msort l =  
  tick 1.0; match l with  
  | [] -> []  
  | _ :: _ ->  
    let l1, l2 = split l in  
    merge (msort l1) (msort l2)
```

RaML's bound:

$$1 - 2.5|l| + 3.5|l|^2$$

¹[2]

Resource Decomposition ² by Example - MergeSort

Resource Decomposition ² by Example - MergeSort

1. Identify the semantic property RaML struggles to bound tightly. For MergeSort, this is its recursion depth.

Resource Decomposition ² by Example - MergeSort

1. Identify the semantic property RaML struggles to bound tightly. For MergeSort, this is its recursion depth.
2. Transform MergeSort into taking its recursion depth d as an argument:

```
let rec msort' l (d : nat) =  
  match d with 0 -> raise exception | Succ(d') ->  
    tick 1.0; match l with  
    | [] -> []  
    | _ :: _ ->  
      let l1, l2 = split l in  
      merge (msort' l1 d') (msort' l2 d')
```

Such programs are termed *resource guarded*.

Resource Decomposition ² by Example - MergeSort

1. Identify the semantic property RaML struggles to bound tightly. For MergeSort, this is its recursion depth.
2. Transform MergeSort into taking its recursion depth d as an argument:

```
let rec msort' l (d : nat) =  
  match d with 0 -> raise exception | Succ(d') ->  
    tick 1.0; match l with  
    | [] -> []  
    | _ :: _ ->  
      let l1, l2 = split l in  
      merge (msort' l1 d') (msort' l2 d')
```

Such programs are termed *resource guarded*.

3. Solicit RaML for a bound on msort' . This is $1 + 3.5|l|d$.

Resource Decomposition ² by Example - MergeSort

1. Identify the semantic property RaML struggles to bound tightly. For MergeSort, this is its recursion depth.
2. Transform MergeSort into taking its recursion depth d as an argument:

```
let rec msort' l (d : nat) =  
  match d with 0 -> raise exception | Succ(d') ->  
    tick 1.0; match l with  
    | [] -> []  
    | _ :: _ ->  
      let l1, l2 = split l in  
      merge (msort' l1 d') (msort' l2 d')
```

Such programs are termed *resource guarded*.

3. Solicit RaML for a bound on msort' . This is $1 + 3.5|I|d$.
4. Independently, obtain a **logarithmic** bound b on d in terms of $|I|$ through a data-driven analysis. The final bound on MergeSort is $[b/d](1 + 3.5|I|d) \in O(|I| \log |I|)$.

Resource Decomposition in General

Resource Decomposition in General

1. In theory, we can express cost bounds in terms of arbitrary semantic properties. Our contributions focus on the recursion depth of functions.

Resource Decomposition in General

1. In theory, we can express cost bounds in terms of arbitrary semantic properties. Our contributions focus on the recursion depth of functions.
2. Each semantic property s is defined in terms of the mark_s cost annotation primitive:

```
let rec msort l =  
  markdepth 1; tick 1.0;  
let res = match l with [] -> []  
  _ :: _ ->  
    let l1, l2 = split l in  
    merge (msort l1) (msort l2)  
in markdepth -1; res
```

Such programs are termed *resource decomposed*, and the semantic property s is termed a *resource component*.

Resource Decomposition in General

1. In theory, we can express cost bounds in terms of arbitrary semantic properties. Our contributions focus on the recursion depth of functions.
2. Each semantic property s is defined in terms of the mark_s cost annotation primitive:

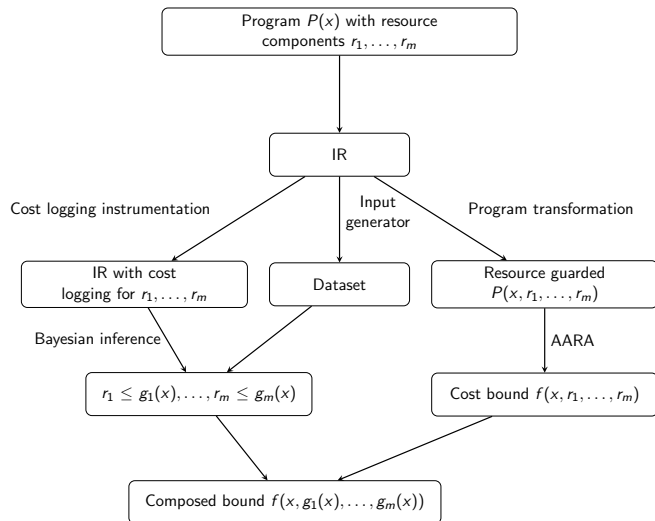
```
let rec msort l =  
  markdepth 1; tick 1.0;  
  let res = match l with [] -> []  
    _ :: _ ->  
      let l1, l2 = split l in  
        merge (msort l1) (msort l2)  
in markdepth -1; res
```

Such programs are termed *resource decomposed*, and the semantic property s is termed a *resource component*.

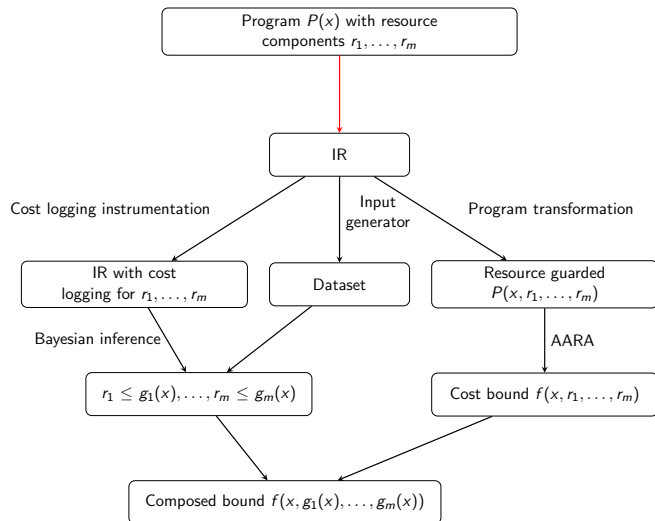
3. Given a cost dataset, recursion depth of each mark -annotated function is bounded statistically using *Bayesian inference*.

Our Contribution: Resource Decomposition for RaML 2

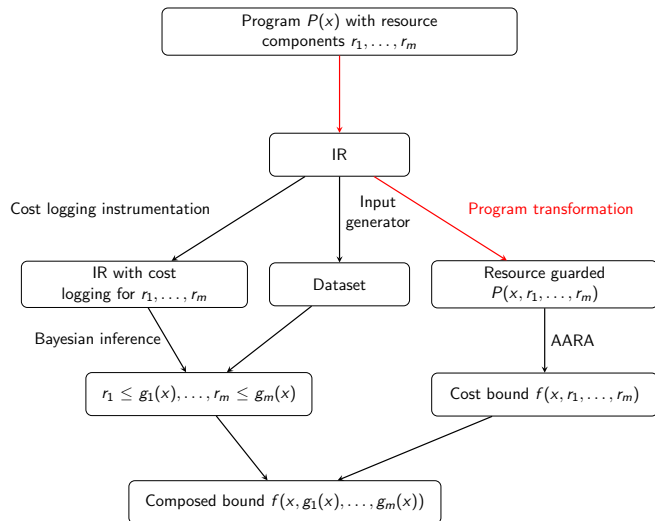
Our Contribution: Resource Decomposition for RaML 2



Our Contribution: Resource Decomposition for RaML 2



Our Contribution: Resource Decomposition for RaML 2



Resource Decomposed to Resource Guarded

We term mark_s as a *resource effect*, and expressions performing it as *resource effectful*. Our IR does not make a type distinction between pure and effectful expressions.

Resource Decomposed to Resource Guarded

We term mark_s as a *resource effect*, and expressions performing it as *resource effectful*. Our IR does not make a type distinction between pure and effectful expressions.

$$\overline{\Gamma \vdash \text{mark}_s(z \in \mathbb{Z}) : 1}$$

Resource Decomposed to Resource Guarded

We term mark_s as a *resource effect*, and expressions performing it as *resource effectful*. Our IR does not make a type distinction between pure and effectful expressions.

$$\overline{\Gamma \vdash \text{mark}_s(z \in \mathbb{Z}) : 1}$$

Thus, we introduce the monadic type $F(\tau)$ and lift all resource effectful expressions into it.

Resource Decomposed to Resource Guarded

We term mark_s as a *resource effect*, and expressions performing it as *resource effectful*. Our IR does not make a type distinction between pure and effectful expressions.

$$\overline{\Gamma \vdash \text{mark}_s(z \in \mathbb{Z}) : 1}$$

Thus, we introduce the monadic type $F(\tau)$ and lift all resource effectful expressions into it.

$$\overline{\Gamma \vdash \text{mark}_s(z \in \mathbb{Z}) : F(1)}$$

Resource Decomposed to Resource Guarded

We term mark_s as a *resource effect*, and expressions performing it as *resource effectful*. Our IR does not make a type distinction between pure and effectful expressions.

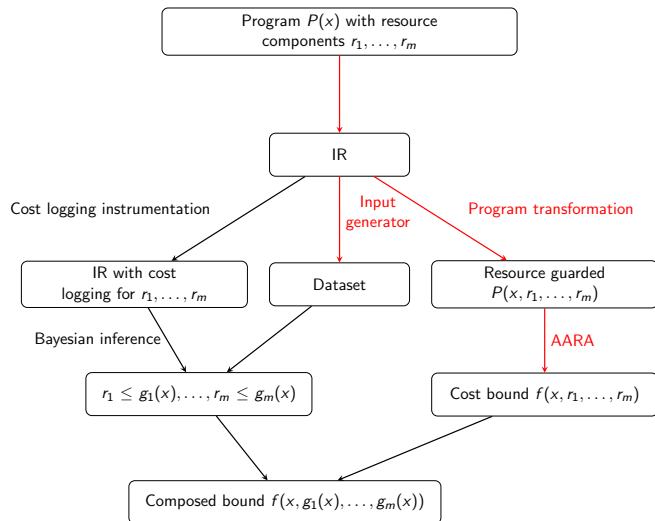
$$\overline{\Gamma \vdash \text{mark}_s(z \in \mathbb{Z}) : 1}$$

Thus, we introduce the monadic type $F(\tau)$ and lift all resource effectful expressions into it.

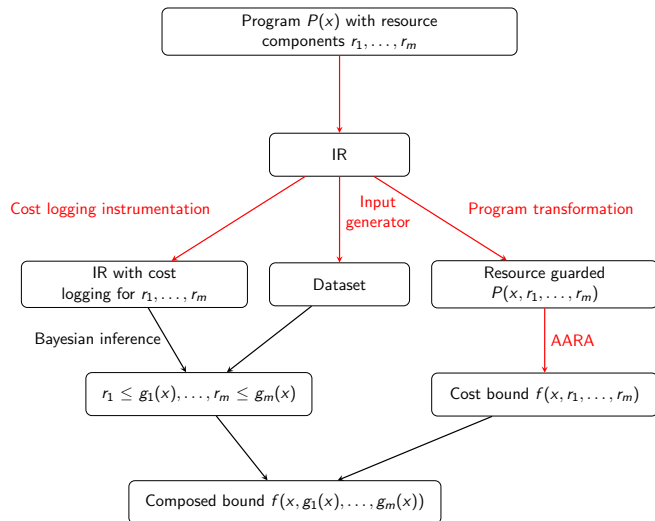
$$\overline{\Gamma \vdash \text{mark}_s(z \in \mathbb{Z}) : F(1)}$$

Translate as $\llbracket F(\tau) \rrbracket = \sigma \rightarrow \sigma \times \llbracket \tau \rrbracket$, where $\sigma = \text{nat}^n$ if resource decomposed program contains n resource components.

Our Contribution: Resource Decomposition for RaML 2



Our Contribution: Resource Decomposition for RaML 2



Cost logging instrumentation

Let $f_i : \tau_1 \rightarrow \cdots \rightarrow \tau_n \rightarrow \rho$ be a function whose recursion depth we treat as resource component i .

Cost logging instrumentation

Let $f_i : \tau_1 \rightarrow \cdots \rightarrow \tau_n \rightarrow \rho$ be a function whose recursion depth we treat as resource component i .

User must define a *size function* $s_i : \tau_i \rightarrow \text{nat}^k$ for each argument type τ_i .

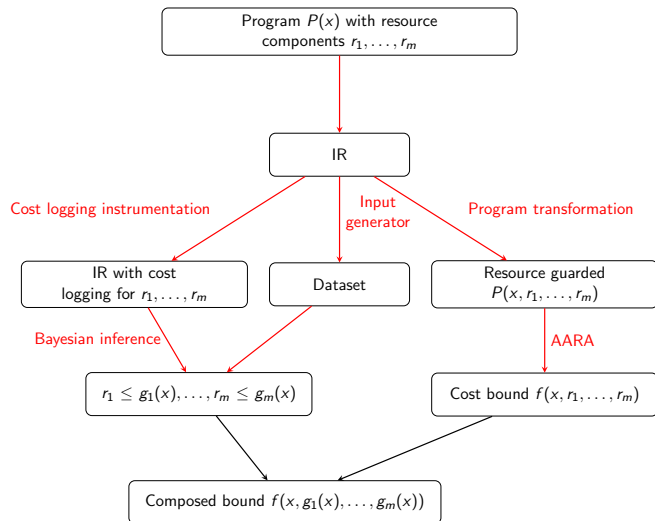
Cost logging instrumentation

Let $f_i : \tau_1 \rightarrow \cdots \rightarrow \tau_n \rightarrow \rho$ be a function whose recursion depth we treat as resource component i .

User must define a *size function* $s_i : \tau_i \rightarrow \text{nat}^k$ for each argument type τ_i .

Instrument f to record its recursion depth and the sizes of its arguments $x_{1\dots n}$ as $\langle s_1(x_1), s_2(x_2), \dots, s_n(x_n) \rangle$ every time it is called.

Our Contribution: Resource Decomposition for RaML 2



Local V/s. Global Recursion Depth Bounds

We run the instrumented IR on our randomly generated set of inputs.

Local V/s. Global Recursion Depth Bounds

We run the instrumented IR on our randomly generated set of inputs.

This induces a cost dataset $\mathcal{D}$ such that $\mathcal{D}_{f_i}[j]$ represents the sizes of f_i 's arguments and its peak recursion depth for the j^{th} time it is called.

Local V/s. Global Recursion Depth Bounds

We run the instrumented IR on our randomly generated set of inputs.

This induces a cost dataset $\mathcal{D}$ such that $\mathcal{D}_{f_i}[j]$ represents the sizes of f_i 's arguments and its peak recursion depth for the j^{th} time it is called.

Bayesian inference yields a *univariate* bound on the recursion depth of f_i in terms of the sizes of its *own* (local) arguments.

Local V/s. Global Recursion Depth Bounds

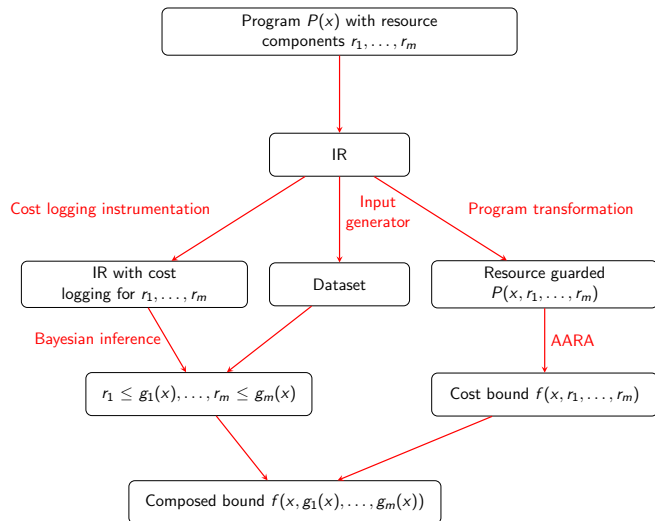
We run the instrumented IR on our randomly generated set of inputs.

This induces a cost dataset $\mathcal{D}$ such that $\mathcal{D}_{f_i}[j]$ represents the sizes of f_i 's arguments and its peak recursion depth for the j^{th} time it is called.

Bayesian inference yields a *univariate* bound on the recursion depth of f_i in terms of the sizes of its *own* (local) arguments.

No method to automatically upper bound argument sizes to f_i in terms of global argument size (yet).

Our Contribution: Resource Decomposition for RaML 2



Results

Benchmark	Ground Truth	Plain AARA	Resource Decomposition
MergeSort	$O(x \log x)$	$1.5 x ^2 - 1.5 x $	$O(x \log x)$
HeapInsert	$O(x \log x)$	$0.5 x ^2 + 0.5 x $	$O(x \log x)$
BalancedBST	$O((x_1 + x_2) \log x_1)$	$0.4 x_1 + x_1 x_2 + 0.7 x_1 ^2 + x_2 $	$O((x_1 + x_2) \log x_1)$
RedBlackTree	$O((x_1 + x_2) \log x_1)$	$1.5 x_1 + x_1 x_2 + 0.5 x_1 ^2 + x_2 $	$O((x_1 + x_2) \log x_1)$
RedBlackJoin	$O(\log x_1 + \log x_2)$	$ x_1 + x_2 $	$O(\log x_1 + \log x_2)$

Results

Benchmark	Ground Truth	Plain AARA	Resource Decomposition
MergeSort	$O(x \log x)$	$1.5 x ^2 - 1.5 x $	$O(x \log x)$
HeapInsert	$O(x \log x)$	$0.5 x ^2 + 0.5 x $	$O(x \log x)$
BalancedBST	$O((x_1 + x_2) \log x_1)$	$0.4 x_1 + x_1 x_2 + 0.7 x_1 ^2 + x_2 $	$O((x_1 + x_2) \log x_1)$
RedBlackTree	$O((x_1 + x_2) \log x_1)$	$1.5 x_1 + x_1 x_2 + 0.5 x_1 ^2 + x_2 $	$O((x_1 + x_2) \log x_1)$
RedBlackJoin	$O(\log x_1 + \log x_2)$	$ x_1 + x_2 $	$O(\log x_1 + \log x_2)$

Benchmark	Resource Decomposition		
	Bayesian Inference	Size Bound	Guarded AARA
MergeSort	$d_1 \in O(\log l)$	$ l \leq x $	$2 x d_1$
HeapInsert	$d_1 \in O(\log(h_1 + h_2))$	$ h_1 + h_2 \leq x $	$ x + x d_1$
BalancedBST	$d_1 \in O(\log l)$ $d_2 \in O(\log t)$	$ l \leq x_1 $ $ t \leq x_1 $	$1 + 0.5d_1 x_1 + 0.5 x_1 + x_2 d_2$
RedBlackTree	$d_1 \in O(\log t)$ $d_2 \in O(\log t)$	$ t \leq x_1 $ $ t \leq x_1 $	$ x_1 d_1 + x_1 + x_2 d_2 + x_2 $
RedBlackJoin	$d_i \in O(\log(t_1 + t_2))$	$ t_1 + t_2 \leq x_1 + x_2 $	$d_1 + d_2$

References I

-  L. Pham, Y. Niu, N. Glover, F. Saad, and J. Hoffmann.
Integrating resource analyses via resource decomposition.
Proc. ACM Program. Lang., 9(OOPSLA2), Oct. 2025.
-  D. Wang, D. M. Kahn, and J. Hoffmann.
Resource aware ml, 2020.